SASD GmbH

downloads-sasd Entwicklerdokumentation

Projektanlage, Entwicklungsumgebung, Composer-Pakete, Testen, API-Dokumentation und sichere Weiterentwicklung

Dokumenttyp	Technische Entwicklerdokumentation
Projekt	downloads-sasd / downloads.sasd.de
Version	v0.1.0
Status	Arbeitsgrundlage fuer Weiterentwicklung und Wartung
Datum	2026-03-31

Zweck dieses Dokuments: Diese Dokumentation soll Ihnen helfen, das Projekt fachlich und technisch zu verstehen, eine saubere lokale Entwicklungsumgebung einzurichten, Composer-Dev-Tools nachzuziehen, Tests und API-Dokumentation einzufuehren und das System kontrolliert weiterzuentwickeln.

1. Ziel, Scope und Leserschaft

Dieses Dokument ist die zentrale Entwicklerhilfe fuer das Repository downloads-sasd. Es erklaert, wie das Projekt aufgebaut ist, wie es lokal gestartet wird, welche Composer-Ergaenzungen sinnvoll sind und wie neue Funktionen in die bestehende Architektur eingepasst werden sollten. Der Text ist bewusst ausfuehrlich formuliert, damit Sie auch nach laengerer Pause ohne langes Einlesen wieder produktiv werden.

Die Dokumentation richtet sich in der aktuellen Phase primaer an Sie selbst als Maintainer. Gleichzeitig ist sie so strukturiert, dass spaetere Entwickler oder externe Unterstuetzung dieselbe Arbeitsgrundlage nutzen koennen.

- Entwicklerperspektive statt Endanwenderperspektive.
- Schwerpunkt auf Verstehen, Nachvollziehen und sauberem Weiterentwickeln.
- Keine Framework-Magie, sondern nachvollziehbare Architektur und konkrete Arbeitsablaeufe.

2. Architekturidee in einem Satz

downloads-sasd ist eine leichte PHP-Webanwendung mit Front Controller, Request/Response- und Routing-Schicht, Controller-Ebene, Service-Schicht, Repository-Abstraktion, View-Renderer und einer dateibasierten JSON-Katalogquelle, die spaeter gegen eine Datenbank ausgetauscht werden kann.

Architekturprinzipien

Prinzip	Konkrete Bedeutung im Projekt
Kleine robuste V1	Das Projekt verzichtet bewusst auf ein schweres Framework, um einen transparenten und kontrollierbaren Start zu ermoeglichen.
Dateibasiert zuerst	Produkte und Artefakte stammen aus JSON-Quellen; dadurch entfallen Datenbank, Migrationen und Admin-UI in der ersten Phase.
Repository-Schicht	Controller und Views kennen keine Dateiformate und keine Verzeichnisdetails.
I18n von Anfang an	UI-Texte liegen in Sprachdateien, Inhaltsdaten sind fuer mehrere Sprachen vorgesehen.
Migration ohne Neustart	Ein kuenftiger Datenbankumzug soll vor allem Repository-Implementierungen betreffen und nicht das gesamte System.

3. Entwicklungsumgebung und erster Start

3.1 Pflichtvoraussetzungen

Lokale Mindestvoraussetzungen

Komponente	Warum sie benoetigt wird	Mindestempfehlung
PHP	Ausfuehrung der Webanwendung und der CLI-Skripte.	8.2 oder neuer
Git	Checkout, Branching und nachvollziehbare Aenderungshistorie.	aktueller Stand
Editor/IDE	Lesbarer Zugriff auf PHP, JSON,	VS Code oder aehnlich

	Markdown und Shell-Dateien.	
Terminal	Start des lokalen Servers, Composer, Tests und Doku-Generierung.	Shell Ihrer Wahl

3.2 Optional, aber fuer echte Weiterentwicklung sehr sinnvoll

- Composer fuer Dev-Abhaengigkeiten, standardisierte Workflows und spaetere Paketpflege.
- PHPUnit fuer automatisierte Tests.
- phpDocumentor fuer API-Dokumentation.
- Xdebug fuer Step-Debugging.
- Optional ein statisches Analysewerkzeug wie PHPStan, sobald die Codebasis waechst.

3.3 Repository holen und lokal starten

```
git clone https://github.com/Robin-Goerlach/downloads-sasd.git
cd downloads-sasd
php -S 127.0.0.1:8080 -t public
```

Die Anwendung ist danach lokal erreichbar. Weil Bootstrap einen eigenen App-Autoloader registriert, funktioniert dieser Start bereits ohne Composer.

3.4 Was beim Start technisch passiert

- public/index.php ist der Front Controller und der einzige oeffentliche Einstiegspunkt.
- Bootstrap laedt Konfiguration, registriert den App-Autoloader, baut Request, Translator, CatalogLoader, Repositories, Services und Controller zusammen.
- Der Router ordnet den Request einer Route zu und uebergibt an den passenden Controller.
- Der Controller holt Daten ueber den CatalogService und rendert ueber View ein HTML-Template.

4. Composer bewusst einfuehren und erweitern

Das Repository startet derzeit auch ohne vendor-Verzeichnis. Fuer nachhaltige Weiterentwicklung sollte trotzdem Composer genutzt werden. So entstehen eine reproduzierbare lokale Umgebung, klar definierte Dev-Werkzeuge und spaeter auch ein sauberer Weg fuer Autoload-Erweiterungen und Skripte.

4.1 Sinnvolle erste Dev-Pakete

Empfohlene Composer-Pakete

Paket	Rolle	Empfehlung fuer downloads-sasd
phpunit/phpunit	Testframework	Fuer Unit-Tests der Service-, Repository- und Hilfsklassen.
phpdocumentor/shim	API-Dokumentation	Pragmatischer Einstieg fuer phpDocumentor im Projektkontext.
phpstan/phpstan	Statische Analyse	Optional, aber spaeter sehr wertvoll fuer Architekturdisziplin.
friendsofphp/php-cs-fixer	Code-Formatierung	Optional zur Vereinheitlichung des PHP-

Stils.

4.2 Empfohlene Befehle

```
composer require --dev phpunit/phpunit
composer require --dev phpdocumentor/shim
composer require --dev phpstan/phpstan
composer require --dev friendsofphp/php-cs-fixer
```

Wenn Sie bewusst klein starten moechten, genuegen zuerst PHPUnit und phpDocumentor. Statische Analyse und Formatierung koennen im zweiten Schritt folgen.

4.3 Composer-Skripte vorschlagen

```
{
  "scripts": {
    "test": "phpunit",
    "doc:api": "phpdoc run -d src -t docs/api",
    "catalog:validate": "php cli/validate-catalog.php",
    "catalog:rebuild": "php cli/rebuild-catalog.php"
  }
}
```

Die bestehende composer.json ist aktuell minimal. Sie kann ohne Architekturschaden in diese Richtung erweitert werden, sobald die ersten Dev-Abhaengigkeiten gepflegt werden sollen.

5. Verzeichnisstruktur lesen und verstehen

Projektstruktur und Verantwortlichkeiten (Teil 1)

Pfad	Aufgabe	Worauf Entwickler achten sollten
public/	Front Controller und oeffentliche Dateien.	Kein Domaenenwissen in diese Schicht kippen; nur Web-Einstiegspunkt und Assets.
app/	Bootstrap und zentrale Verdrahtung.	Hier entsteht die Anwendung; keine Fachlogik verteilen.
src/Http/	Request, Response, Router und Controller.	HTTP-nahe Aufgaben sauber von Kataloglogik trennen.
src/Repository/	Interfaces und JSON-Repositories.	Diese Schicht ist spaeter der Hauptansatzpunkt fuer einen Datenbankumzug.
src/Service/	Anwendungslogik und Such-/Filterverhalten.	Hier lebt die eigentliche Kataloglogik.
src/View/	View-Renderer und Template-Helfer.	Keine Dateisystem- oder Suchlogik in Templates verlagern.

Projektstruktur und Verantwortlichkeiten (Teil 2)

Pfad	Aufgabe	Worauf Entwickler achten sollten
config/	App-Konfiguration inklusive aktivierter Sprachen und Pfade.	Relative Pfade bleiben lesbar; Bootstrap macht sie absolut.
resources/lang/	Sprachdateien fuer UI-Texte.	Neue UI-Texte immer hier sauber ergaenzen.
data/catalog/	Quell- und generierte Katalogdaten.	Bearbeiten Sie Quell-Daten, nicht die generierten Dateien.
cli/	Wartungs- und Build-Skripte.	Hier gehoeren wiederholbare technische Aufgaben hinein.
docs/	Projektnahe Dokumentation und Screenshots.	Technische Begleitdokumente strukturiert halten.

6. Der Request- und Renderfluss im Projekt

6.1 Einstieg ueber public/index.php

public/index.php ist klein gehalten und genau deshalb wichtig. Die Datei laedt Bootstrap, initialisiert die Anwendung und faengt Fehler ab. Das ist wuenschenswert, denn der oeffentliche Einstiegspunkt bleibt dadurch lesbar und kontrollierbar.

6.2 Bootstrap als Verdrahtungszentrum

- Konfiguration laden und Pfade aufloesen.
- Eigenen App-Autoloader registrieren.
- Request aus den PHP-Globals erzeugen.
- LocaleResolver und Translator initialisieren.
- CatalogLoader und darauf aufbauende JSON-Repositories erstellen.
- CatalogService und View bauen.
- Controller instanziieren und Routen registrieren.

6.3 Router und Controller

Der Router kennt nur HTTP-Methode, Pfad und Handler. Er ist absichtlich klein und unterstuetzt Platzhalter wie /products/{slug}. Diese Einfachheit ist fuer das aktuelle Projekt ein Vorteil, weil sie Verhalten transparent macht. Controller sollten ebenfalls klein bleiben: Request validieren, Service aufrufen, View rendern oder Download ausliefern.

6.4 Service- und Repository-Schicht

CatalogService sammelt die such- und darstellungsnahe Logik. Repositories laden Produkte und Artefakte aus dem CatalogLoader und erzeugen Domain-Objekte. Diese Aufteilung ist die wichtigste Voraussetzung dafuer, spaeter von JSON auf eine Datenbank zu wechseln, ohne alle oberen Schichten neu zu schreiben.

7. Datenmodell und Kataloglogik

Die aktuelle Datenquelle ist der JSON-Katalog. Das ist kein Notbehelf, sondern eine bewusst gewaehlte Startarchitektur. Wichtig ist, dass die Anwendung nicht ueberall direkt auf Dateiformate zugreift. Stattdessen kapseln CatalogLoader und Repositories den Zugriff.

Zentrale Datenebenen

Ebene	Beschreibung
Source JSON	Die editierbaren Quelldateien fuer Produkte und Artefakte.
Generated Catalog	Zusammengefuehrte und validierte Katalogdarstellung fuer die Laufzeit.
Repository-Objekte	Produkt- und Artefaktobjekte, mit denen Services arbeiten.
View-Modell	Die fuer Templates benoetigten Daten, aufbereitet durch Service und View.

7.1 Warum diese Trennung spaeter hilft

- Suche, Filterung und Sortierung bleiben im Wesentlichen erhalten, auch wenn die Datenquelle spaeter nicht mehr JSON ist.
- Controller und Templates bleiben weitgehend stabil, solange Interfaces und Rueckgabeobjekte konsistent bleiben.
- Migrationsrisiken sinken, weil Datenzugriff nicht im gesamten Projekt verstreut ist.

8. Mehrsprachigkeit weiterdenken

Die erste Version unterstuetzt Deutsch und Englisch. Intern ist das System so vorbereitet, dass weitere Sprachen spaeter strukturiert hinzugefuegt werden koennen. Entwickler sollten zwischen UI-Texten und Inhaltsdaten unterscheiden.

Mehrsprachigkeit im Projekt

Bereich	Ablageort	Entwicklerregel
UI-Texte	resources/lang/*.php	Jeder neue sichtbare UI-Text bekommt einen stabilen Schluessel statt harter Strings in Templates.
Produkt- und Artefaktinhalte	Katalogdaten	Titel, Summary und Beschreibung sprachspezifisch pflegen.
Sprachwahl	config/app.php und LocaleResolver	Neue Sprachen erst aktivieren, wenn UI-Datei und Inhaltsdaten vorbereitet sind.

8.1 Praktische Regel

Wenn Sie neue Funktionen entwickeln, behandeln Sie Texte immer so, als waeren mindestens Deutsch und Englisch Pflicht. Das vermeidet spaetere kostspielige Nacharbeiten.

9. Tests einfuehren und sinnvoll strukturieren

Automatisierte Tests sind fuer dieses Projekt besonders wertvoll, weil die Architektur bewusst leichtgewichtig ist. Je weniger Framework-Schutz vorhanden ist, desto wichtiger ist eine kontrollierte Pruefung der eigenen Logik.

9.1 Was zuerst getestet werden sollte

- CatalogService: Suche, Filterung, Sortierung, Gruppierungs- und Latest-Logik.
- JsonProductRepository und JsonArtifactRepository: korrektes Laden und Aufbauen der Domain-Objekte.
- LocaleResolver und Translator: Sprachauflosung und Fallback-Verhalten.
- CatalogLoader: Validierung und Fehlerfaelle bei ungueltigen JSON-Daten.

9.2 Vorgeschlagene Teststruktur

```
tests/
  Unit/
    Service/
    Repository/
    Localization/
  Integration/
    Catalog/
```

9.3 Minimaler PHPUnit-Start

```
vendor/bin/phpunit
```

Sobald eine phpunit.xml eingefuehrt wird, sollten Test-Suites und gegebenenfalls Bootstrap-Dateien explizit hinterlegt werden. Beginnen Sie klein, aber testen Sie die Kernlogik frueh.

10. API-Dokumentation mit phpDocumentor

Weil das Projekt bereits ueber viele DocBlocks verfuegt und Sie explizit gut kommentierten Code wuenschen, ist phpDocumentor eine sehr passende Ergaenzung. Es hilft, Klassen, Methoden, Parameter und Rueckgabetypen projektweit sichtbar und nachvollziehbar zu machen.

10.1 Dokumentation erzeugen

```
vendor/bin/phpdoc run -d src -t docs/api
```

10.2 Worauf bei DocBlocks zu achten ist

- Klassen-DocBlocks beschreiben Zweck, Verantwortungsgrenzen und wichtige Architekturentscheidungen.
- Methoden-DocBlocks sollten Parameter, Rueckgaben und Seiteneffekte beschreiben, wenn sie nicht trivial sind.
- Array-Formate und DTO-aehnliche Strukturen moeglichst praezise dokumentieren.
- Kommentare nicht als Ersatz fuer schlechte Namen missbrauchen; zuerst sauber benennen, dann dokumentieren.

11. Neue Features sauber entwickeln

11.1 Beispiel: neue Filterdimension einfuehren

Wenn Sie spaeter beispielsweise nach Zielgruppe oder Dateigroesse filtern moechten, sollte die Aenderung schichtweise und kontrolliert erfolgen.

- Katalogdatenmodell erweitern, falls neue Metadaten benoetigt werden.
- Repository- und Domain-Objekte so anpassen, dass die neue Information stabil verfuegbar ist.
- CatalogService um Such- und Filterlogik erweitern.
- Controller nur soweit anpassen, wie Request-Parameter gelesen oder weitergereicht werden.
- View und Templates fuer die neue Auswahl oder Anzeige ergaenzen.
- Tests und Dokumentation nachziehen.

11.2 Beispiel: Datenbank spaeter einfuehren

Beim Umstieg auf eine Datenbank sollte nicht die gesamte Anwendung umgeschrieben werden. Stattdessen bleiben Request-Fluss, Controller, View und grosse Teile des CatalogService moeglichst stabil. Sie fuehren zunaechst neue Repository-Implementierungen ein und schalten dann schrittweise um.

12. Coding-Konventionen und Kommentierstil

Dieses Projekt soll spaeter gut lesbar sein. Deshalb sind bewusst zwei Kommentar-Ebenen sinnvoll: DocBlocks fuer API-Dokumentation und Architekturverstaendnis sowie kurze //-Kommentare fuer lokale Gedankengaenge, Spezialfaelle oder nicht sofort offensichtliche Entscheidungen im Code.

Empfohlener Kommentierstil

Kommentarart	Wann verwenden	Beispielzweck
/** ... */	Klassen, Methoden, wichtige Eigenschaften	Vertrag, Zweck, Parameter, Rueckgabe, Architekturkontext
// ...	lokale Erlaeuterungen im Code	Warum eine Schleife, ein Fallback oder ein Sonderfall so gebaut wurde
Keine Kommentare	triviale oder selbsterklaerende Zeilen	Nicht jeden offensichtlichen Ausdruck kommentieren

12.1 Namensgebung

- Klassen nach Verantwortung benennen, nicht nach Implementierungsdetail.
- Methoden so formulieren, dass ihr Zweck auch ohne Kommentar erkennbar ist.
- String-Schluessel fuer Uebersetzungen stabil und logisch gruppiert halten.
- Slugs, Statuswerte und Gruppenbezeichnungen konsistent halten.

13. Troubleshooting fuer Entwickler

Hauefige Probleme und typische Ursachen

Symptom	Moegliche Ursache	Erster Pruefschritt
Leere Seite oder 500-Fehler	PHP-Fehler in Bootstrap, Template oder Route	PHP-CLI-Startausgabe und Webfehler ansehen; index.php faengt Throwable sichtbar ab.
Artefakt nicht auffindbar	Slug stimmt nicht, Status hidden oder Katalog nicht neu gebaut	Source-JSON pruefen, validieren, generated catalog neu erzeugen.

Sprache wechselt nicht	Locale nicht aktiviert oder Sprachdatei unvollstaendig	config/app.php und resources/lang pruefen.
Download liefert 404	Dateipfad in Katalogdaten falsch oder Datei fehlt	Storage-Pfad und generierte Metadaten gegeneinander pruefen.
Aenderung in JSON wirkt nicht	Generated Catalog veraltet	php cli/validate-catalog.php und php cli/rebuild-catalog.php ausfuehren.

14. Empfohlener taeglicher Entwicklungsworkflow

- Repository aktualisieren und Branch fuer die Aufgabe anlegen.
- Falls noetig Katalogdaten validieren und generierten Katalog neu bauen.
- Lokalen PHP-Server starten.
- Aenderungen schichtweise vornehmen: Datenmodell, Repository, Service, Controller, View.
- Manuell pruefen und, sobald eingefuehrt, Tests ausfuehren.
- DocBlocks und Entwicklerdokumentation nachziehen, wenn die Aenderung konzeptionell relevant ist.
- Commit mit klarer Aussage erstellen; spaeter Changelog und Releases nachziehen.

15. Konkrete naechste technische Schritte fuer downloads-sasd

Aus Entwicklersicht bieten sich fuer die naechsten Iterationen einige besonders wertvolle Ausbaustufen an. Diese Reihenfolge schuetzt die aktuelle Klarheit des Projekts und fuehrt trotzdem schrittweise zu einer professionelleren Plattform.

Naechste Ausbaustufen

Prioritaet	Thema	Begruendung
1	PHPUnit-Basis einfuehren	Die Kernlogik bekommt damit ein Sicherheitsnetz, bevor das Projekt groesser wird.
2	phpDocumentor integrieren	Die vorhandenen DocBlocks werden als API-Dokumentation nutzbar.
3	Composer-Skripte und kleine Dev-Standards	Wiederkehrende Aufgaben werden vereinheitlicht.
4	Interne Upload- oder Redaktionsoberflaeche ohne Datenbank	Pflegekomfort steigt, ohne die Architektur vorschnell zu verkomplizieren.
5	Repository-Tests und spaeter DB-Prototyp	Der kuenftige Migrationspfad wird frueh technisch abgesichert.

Anhang A. Wichtige Befehle auf einen Blick

```
php -S 127.0.0.1:8080 -t public
php cli/validate-catalog.php
```

```
php cli/rebuild-catalog.php
composer require --dev phpunit/phpunit
composer require --dev phpdocumentor/shim
vendor/bin/phpunit
vendor/bin/phpdoc run -d src -t docs/api
```

Anhang B. Kurzcheckliste fuer neue Entwickleraufgaben

- Verstehe zuerst, in welcher Schicht die Aenderung wirklich hingehoert.
- Bearbeite Quell-JSON und nicht nur generierte Daten.
- Fuehre Validierung und Rebuild aus, wenn Katalogdaten betroffen sind.
- Ergaenze UI-Texte ueber Sprachdateien und nicht hart im Template.
- Aendere moeglichst nicht mehrere Schichten chaotisch gleichzeitig.
- Halte DocBlocks und technische Dokumentation konsistent mit dem Code.